

UNIVERSIDAD NACIONAL ABIERTA Y A DISTANCIA

UNAD

Escuela de Ciencias Básicas Tecnología e Ingeniería — ECBTI
Programa: Ingeniería de Sistemas
Curso: Programación | Código: 213023

Sistema Integral de Gestión de Clientes, Servicios y Reservas

Software FJ

Fase 4 — Prácticas Simuladas
Componente Práctico

Nombre	ID / Código
Juan Sebastian Bernal Bermudez	Estudiante
Juan Fernando Capitani Giraldo	Estudiante
Maribel Chacon Garzon	Estudiante
Cristian Camilo Galindo Barragan	Estudiante
Junior David Mancipe Castiblanco	Estudiante

Grupo: 213023_332
Bogotá D.C., Colombia — 2026

1. Introducción

El presente documento constituye la entrega del componente práctico de la Fase 4 del curso de Programación (código 213023) de la Universidad Nacional Abierta y a Distancia. El objetivo central de esta fase es implementar el manejo de excepciones en el desarrollo de aplicaciones orientadas a objetos, buscando estabilidad y robustez en el funcionamiento de las soluciones.

El grupo 213023_332, conformado por cinco estudiantes, desarrolló un sistema integral denominado **Software FJ**, orientado a la gestión de clientes, servicios y reservas para una empresa ficticia que ofrece tres tipos de servicios: reservas de salas, alquiler de equipos y asesorías especializadas. El sistema fue construido enteramente en Python, sin uso de bases de datos, empleando programación orientada a objetos y manejo avanzado de excepciones como pilares fundamentales.

El código fue desarrollado de manera colaborativa a través del repositorio de GitHub del grupo, garantizando la participación activa de todos los integrantes mediante commits, ramas individuales y documentación clara de cada módulo.

Objetivos del sistema:

- Implementar los principios de POO: abstracción, herencia, polimorfismo y encapsulación.
- Manejar robustamente errores mediante excepciones personalizadas y bloques try/except, try/except/else, try/except/finally y encadenamiento de excepciones.
- Registrar todos los eventos y errores en un archivo de logs persistente.
- Simular más de 10 operaciones completas, incluyendo casos válidos e inválidos.
- Garantizar que la aplicación nunca se detenga ante errores, manteniéndose estable en todo momento.

2. Repositorio en GitHub

El código fuente completo del proyecto se encuentra disponible en el siguiente repositorio público de GitHub, donde cada integrante del grupo realizó sus aportes mediante ramas y commits individuales:

Repositorio GitHub del Grupo 213023_332

[https://github.com/\[USUARIO_GRUPO\]/software_fj_213023_332](https://github.com/[USUARIO_GRUPO]/software_fj_213023_332)

Nota: Reemplazar [USUARIO_GRUPO] con el usuario de GitHub del equipo antes de la entrega.

Estructura del repositorio:

- `software_fj/main.py` — Punto de entrada — 15 operaciones de simulación
- `software_fj/modelos/cliente.py` — Clase abstracta EntidadBase y clase Cliente
- `software_fj/modelos/reserva.py` — Clase Reserva con ciclo de vida completo
- `software_fj/servicios/servicios.py` — Servicio (abstracta) + 3 servicios especializados
- `software_fj/gestion/gestores.py` — Gestores en memoria para Clientes, Servicios y Reservas
- `software_fj/utils/excepciones.py` — 15 excepciones personalizadas jerarquizadas
- `software_fj/utils/logger.py` — Sistema de logging centralizado con FileHandler
- `README.md` — Documentación completa del proyecto

3. Descripción Técnica del Sistema

3.1 Arquitectura Orientada a Objetos

El sistema implementa una arquitectura modular basada en los cuatro pilares de la Programación Orientada a Objetos:

Abstracción: La clase EntidadBase (ABC) define el contrato base para todas las entidades del sistema. La clase Servicio (ABC) establece la interfaz común para los tres tipos de servicios. Ninguna puede instanciarse directamente.

Herencia: Cliente hereda de EntidadBase. ReservaSala, AlquilerEquipo y AsesoríaEspecializada heredan de Servicio, reutilizando atributos y métodos comunes.

Polimorfismo: Cada servicio implementa `calcular_costo()`, `describir()` y `validar_parametros()` con comportamiento específico. Esto permite tratar todos los servicios de forma uniforme desde los gestores y la clase Reserva.

Encapsulación: Todos los atributos de Cliente y Servicio son privados (`_nombre`, `_email`, etc.). El acceso se realiza exclusivamente mediante `@property` con validación integrada en los setters, lanzando `DatosClienteInvalidosError` ante datos incorrectos.

3.2 Manejo de Excepciones

El sistema implementa una jerarquía completa de excepciones personalizadas, todas heredando de la clase base `SoftwareFJError`, que a su vez hereda de `Exception`. Se utilizaron los siguientes patrones:

Patrón	Ubicación en el código	Propósito
try/except	GestorClientes.registrar(), calcular_costo()	Captura y re-lanza excepciones específicas
try/except/else	Reserva.confirmar()	Bloque else solo se ejecuta si no hubo error
try/except/finally	Reserva.procesar(), OP-14 en main.py	Finally garantiza ejecución limpia siempre
raise ... from e	OP-15, servicios.calcular_costo()	Encadena excepciones preservando causa original

3.3 Sistema de Logging

El módulo `utils/logger.py` implementa un sistema de registro centralizado que escribe simultáneamente en consola (nivel INFO) y en un archivo de texto plano (nivel DEBUG). El archivo se nombra automáticamente con la fecha del día (`software_fj_YYYY-MM-DD.log`) y registra cada operación del sistema con timestamp, nivel de severidad y mensaje descriptivo. Esto garantiza trazabilidad completa sin depender de ningún motor de base de datos.

4. Operaciones de Simulación

El archivo main.py ejecuta 15 operaciones completas que cubren todos los escenarios requeridos por la rúbrica de evaluación:

#	Operación	Resultado
01	Registro de cliente válido (Juan Sebastian Bernal)	Exitoso
02	Registro de 2 clientes válidos (Maribel Chacon, Junior Mancipe)	Exitoso
03	Registro con email inválido ('esto-no-es-un-email')	DatosClienteInvalidosError
3b	Registro de cliente con ID duplicado ('CLI001')	ClienteYaExisteError
04	Creación de 4 servicios válidos (sala, laptop, asesoría, sala 2)	Exitoso
05	Servicio con tarifa negativa (-5000)	ParametroServicioInvalidoError
06	Asesoría con nivel de asesor inválido ('dios')	ParametroServicioInvalidoError
07	Reserva sala 5h + videoconferencia + descuento 5%	\$555,750 COP
08	Reserva sala ejecutiva con 10 personas (máx. 4)	CapacidadExcedidaError
09	Reserva con duración de 0 horas	DuracionInvalidaError
10	Alquiler laptop 8h full-day + seguro + IVA	\$191,173.50 COP
11	Asesoría Expert 3 sesiones virtuales + informe	\$1,396,000 COP
12	Cancelación correcta + intento de doble cancelación	1 ok / ReservaYaCanceladaError
13	Confirmar reserva ya confirmada (RES0001)	ReservaYaConfirmadaError
14	Reserva sobre servicio desactivado (laptop)	ServicioNoDisponibleError
15	Encadenamiento de excepciones (modalidad inválida)	CalculoError from ParametroError

5. Conclusiones

1. El sistema Software FJ demostró que es posible construir aplicaciones robustas y estables sin el uso de bases de datos, gestionando toda la información mediante objetos, listas y diccionarios en memoria, y empleando archivos de texto únicamente para el registro de eventos.
2. La implementación de una jerarquía de excepciones personalizadas (15 clases específicas heredando de SoftwareFJError) permitió identificar y comunicar con precisión cada tipo de error, haciendo el código más legible, depurable y mantenible.
3. El uso de clases abstractas (EntidadBase y Servicio) garantizó un diseño coherente y extensible: agregar un nuevo tipo de servicio solo requiere crear una nueva subclase que implemente los métodos abstractos, sin modificar el código existente.
4. Los bloques try/except/else/finally y el encadenamiento de excepciones demostraron ser herramientas fundamentales para garantizar que la aplicación nunca se interrumpa ante errores, manteniendo el flujo de ejecución bajo control en todo momento.
5. El trabajo colaborativo a través de GitHub, con ramas individuales y commits descriptivos, facilitó la integración del código de los cinco integrantes y permitió llevar un historial claro del avance del proyecto.
6. Esta fase consolidó los fundamentos de la programación orientada a objetos aprendidos durante el curso, llevándolos a un escenario práctico y realista que refleja los desafíos del desarrollo de software profesional.

6. Referencias Bibliográficas

Van Rossum, G., & Drake Jr, F. L. (2024). *El tutorial de Python*. Python Software Foundation. <https://docs.python.org/es/3.12/tutorial/errors.html>

Cuevas Álvarez, A. (2016). *Python 3: curso práctico*. RA-MA Editorial. <https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/106404?page=373> (pp. 373–385)

Romano, F., Baka, B., & Phillips, D. (2019). *Getting Started with Python: Understand Key Data Structures and Use Python in Object-oriented Programming*. Packt Publishing. <https://research-ebSCO-com.bibliotecavirtual.unad.edu.co/linkprocessor/plink?id=b41fd66a-1134-3dcd-8dba-90f36451f08d>

Zambrano, J. P. (2025). *Introducción al uso de GitHub* [Objeto virtual de Información OVI]. Repositorio Institucional UNAD. <https://repository.unad.edu.co/handle/10596/75876>

Silva, F. D. (2025). *Basics of using GitHub* [Objeto virtual de Información OVI]. Repositorio Institucional UNAD. <https://repository.unad.edu.co/handle/10596/75882>